✓ FINAL · LOCKED · v1.0

FINAL-DESC-CMCMIS

Computerized Maintenance & Calibration Management Information System (Simplified)

FINAL PROJECT DESCRIPTION

The canonical pre-build blueprint — everything we are going to build

Field	Value
Project Code	CMCMIS_SIMPLIFIED
Document	FINAL-DESC-CMCMIS
Document Version	v1.0 — FINAL
Document Type	Final Project Description (pre-build)
Domain	Metrology · Calibration · Maintenance · Repair · Registration
Equipment Categories	T&ME (Test & Measurement), F&PE (Fabrication & Production)
Environment	High-precision engineering, ISRO SAC-like (government / defence-grade)
Project Type	Industry-grade active-user production system
Prepared by	Deep Sorathiya (DS), Software Developer Intern
Build Mode	Solo developer + AI engineering pair
Sprint Length	10 weeks (MVP) · production-grade quality target
Phases Completed	Phase 0 (Discovery) · Phase 1 (Modeling) · Phase 2 (Architecture)
Locked Decisions	20 master decisions + 6 stack additions + 5 phase confirmations
Next Phase	Phase 3 — Database Design (begins with existing schema audit)

Field	Value
Status	FINAL — LOCKED — READY TO BUILD

AUTHORITY OF THIS DOCUMENT

This is the canonical, locked, single-source-of-truth description of CMCMIS_SIMPLIFIED prior to construction. It supersedes every earlier interim draft. The decisions, rules, modules, and architectural choices captured here are the contract for the 10-week build sprint. Any change to anything in this document requires an explicit revision with a new version number.

Table of Contents

Executive Summary

CMCMIS_SIMPLIFIED is an industry-grade web information system that digitizes the end-to-end lifecycle of test/measurement and production equipment in a high-precision engineering laboratory. It replaces ad-hoc tracking — spreadsheets, paper forms, manual follow-ups — with a role-aware, auditable, single source of truth used daily by engineers, lab leads, and a single tier of administrators.

Three of nine modules constitute the 10-week Minimum Viable Product: Authentication & RBAC; Equipment master with registration and verification; Job Requests + Job Cards (the operational core); Dashboard + Inquiry (the visibility layer). The remaining modules (Schedule, Procurement, Reports, Master Data Management UI, Notifications) are deferred to Phase 2 post-handoff but their data is already modelled in the schema.

The system is built as a 3-tier monolith — React 18 SPA on Vite + Tailwind, Node.js + Express REST API, MySQL 8.x with phpMyAdmin — deployed entirely on internal on-premises infrastructure. No public cloud, no external SMTP, no file storage, no Redis. The only thing entering or leaving the system is HTTPS traffic between the browser and the API. PDFs (job cards, calibration certificates) are generated on demand from current database state and streamed to the browser, never persisted.

This document captures the project at its most decision-dense moment — the day before construction begins. All 20 master decisions are locked, all 6 stack additions are locked, all 5 phase confirmations are closed. The only outstanding items are domain-specific inputs (existing 64-table database dump, two Super Admin employee IDs, table usage classification) which are scheduled for delivery on Day 1 of Phase 3.

Status Snapshot

Phase	Status	Outcome
Phase 0 — Discovery & Requirements Capture	Complete	23 clarifying questions resolved; domain captured; 17 critical gaps surfaced
Phase 1 — Domain Modeling & Ideation	Complete	5 personas, 3 critical journeys, 36+ business rules, 45 functional requirements, MVP scope locked
Phase 2 — System Architecture & Tech Stack	Complete	3-tier monolith, full stack chosen, 20 master decisions locked, 6 stack additions locked
Phase 3 — Database Design (~70 tables)	Begins Day 1 after document delivery	Existing schema audit → target schema reconciliation → CREATE TABLE scripts → seeders
Phases 4 → 9	Sequenced	API contracts → Module build → Hardening → Scaling → Deployment → Handover

Part I — Foundation

1. Project Definition

1.1 The One-Sentence Version (Simple)

> **INFO**
> CMCMIS is the system that knows where every instrument in the labs is, what state it is in, when it needs calibration, who is working on it, and what was done to it — for as long as the instrument exists.

1.2 The Domain in Five Words

```
REGISTER → CALIBRATE → MAINTAIN → REPAIR → RETIRE
```

Every record in CMCMIS_SIMPLIFIED is one event on one instrument's journey through these five phases. The schema, state machines, and user interface all serve this single narrative.

1.3 The Strategic Picture (Deep)

CMCMIS_SIMPLIFIED is not a CRUD application. In a high-precision engineering environment, data integrity is a regulatory matter. The system therefore operates across five strategic layers, each with non-negotiable correctness requirements.

Layer	What Lives Here	Why It Matters
People	5 roles, hundreds of users over the system's lifetime	Wrong access leads to data corruption or compliance breach
Process	Three strict state machines (Job Request, Job Card, Equipment)	Wrong transitions cause audit failures — government-grade orgs cannot tolerate this
Assets	Thousands of instruments plus their full history	Lose history = lose calibration provenance = unsafe measurements downstream
Money	Purchase orders, spares, vendor contracts (Phase 2)	Wrong PO chains cause procurement audit issues
Knowledge	Reports, calibration certificates, signatures	This is the institutional memory of the laboratory

1.4 Engagement Context

Aspect	Decision
Organisation context	ISRO SAC-like government / technical / defence-grade environment
Project status	Real organisational production system (not a prototype, not a college project)
Owner	Solo Software Developer Intern (with AI engineering pair)
Timeline	10 weeks for MVP; full go-live after testing + stakeholder approval

Aspect	Decision
Deployment	Internal on-prem / private infrastructure (not public cloud)
Existing system	Approximately 64 tables already exist with records — reviewed in Phase 1 of construction
Quality bar	Industry-grade; defence-grade context; no compromise on auditability

2. Final Master Decision Ledger

Every locked decision is captured here with its identifier so it can be referenced in code reviews, design discussions, and audits. New decisions extend the ledger; existing decisions are not retroactively changed without a new ID.

2.1 Architectural & Stack Decisions (D1–D20)

#	Decision	Status
D1	JavaScript + JSDoc + Zod (no TypeScript in v1)	✓ LOCKED
D2	Raw SQL via mysql2/promise + Repository Pattern (no ORM)	✓ LOCKED
D3	React Query for server state	✓ LOCKED
D4	Zustand for client state (over Redux)	✓ LOCKED
D5	pdfkit for PDF generation (over Puppeteer)	✓ LOCKED
D6	Pino for logging (over Winston)	✓ LOCKED
D7	Layered backend: Routes → Controllers → Services → Repositories	✓ LOCKED
D8	Feature-based frontend folder structure (not layer-based)	✓ LOCKED
D9	Nginx reverse proxy in production deployment	✓ LOCKED
D10	New equipment defaults to PENDING_VERIFICATION	✓ LOCKED
D11	At least 2 Super Admin employee IDs seeded via env CSV	✓ LOCKED
D12	Five roles only — Admin role removed, absorbed by Super Admin	✓ LOCKED
D13	Equipment registration open to four roles (not View-Only)	✓ LOCKED
D14	Equipment verification belongs to Lab In-Charge + Super Admin	✓ LOCKED
D15	Master data CRUD belongs to Super Admin only	✓ LOCKED
D16	Lookup values seeded in Week 2; UI deferred to Phase 2	✓ LOCKED
D17	JWT 15 minutes + refresh cookie 7 days, httpOnly	✓ LOCKED
D18	Bootstrap via .env SUPER_ADMIN_EMPLOYEE_IDS CSV	✓ LOCKED

#	Decision	Status
D19	All PDFs streamed, never buffered fully	✓ LOCKED
D20	Audit log on every state-changing operation	✓ LOCKED

2.2 Stack Additions (S1–S6) — All Locked

#	Addition	Reason	Status
S1	cookie-parser (BE)	Required to read httpOnly refresh cookie on /auth/refresh	✓ LOCKED
S2	compression (BE)	gzip/brotli responses — supports NFR p95 < 500 ms target	✓ LOCKED
S3	CSRF token on /auth/refresh	NFR mandates CSRF defence; custom double-submit token approach	✓ LOCKED
S4	@tanstack/react-table (FE)	Headless table primitive for equipment / job lists	✓ LOCKED
S5	@tailwindcss/forms (FE)	Sane default form styling on top of Tailwind base	✓ LOCKED
S6	vitest + supertest	Unit testing (vitest) + API integration testing (supertest) — auth & state machines	✓ LOCKED

2.3 Phase Confirmations (C1–C5) — All Closed

#	Confirmation	Status
C1	Five roles final: Super Admin, Lab In-Charge, Lab Engineer, Normal User, View-Only	✓ CONFIRMED
C2	Master Data Management remains Phase 2, accessible by Super Admin only (build instructions to follow)	✓ CONFIRMED
C3	Lookup data (vendors, equipment types, divisions) seeded Week 2; edited via phpMyAdmin during MVP	✓ CONFIRMED
C4	Bootstrap with at least 2 Super Admin employee IDs (IDs delivered with Phase 3 inputs)	✓ CONFIRMED
C5	Equipment verification (PENDING → ACTIVE) belongs to Lab In-Charge + Super Admin	✓ CONFIRMED

✓ LOCKED

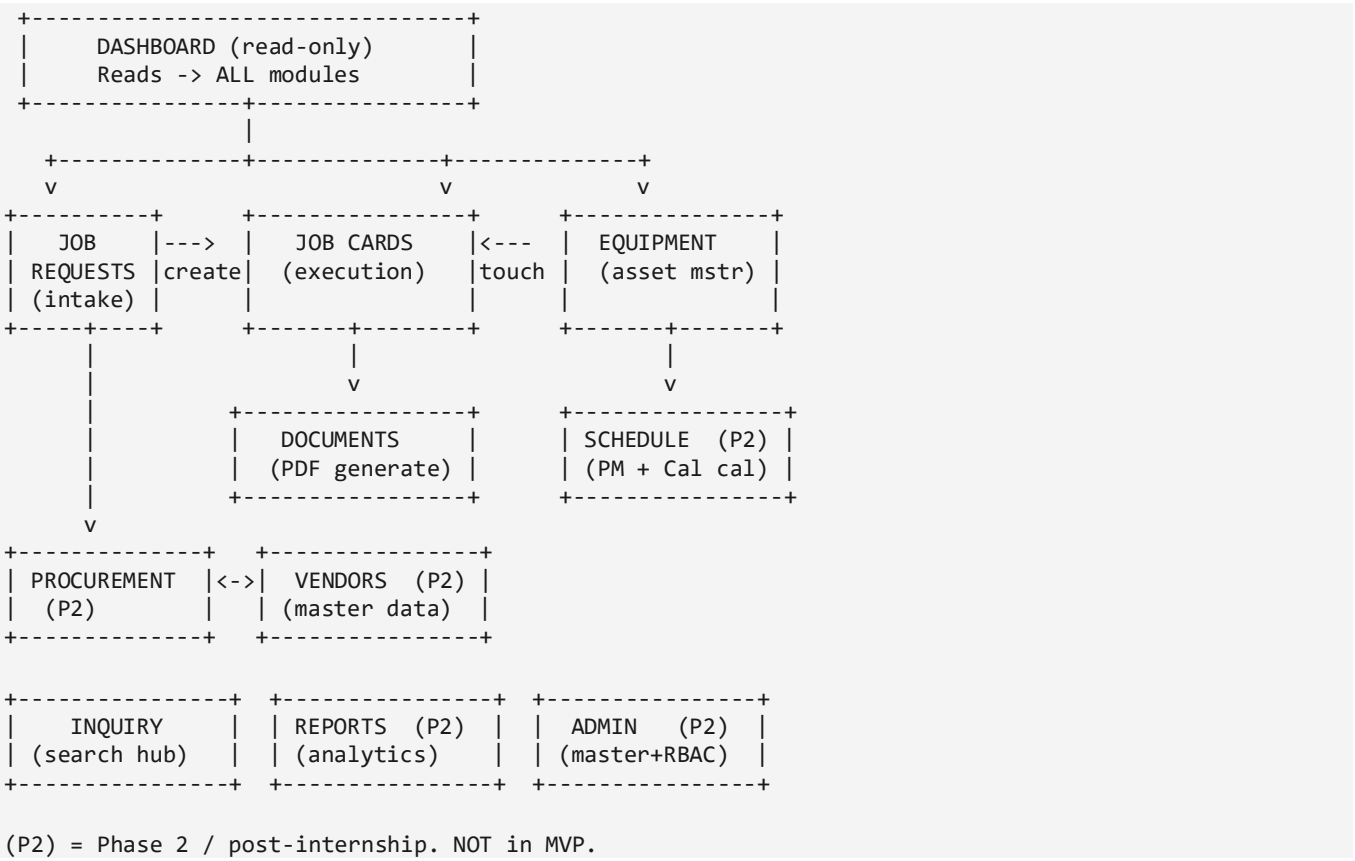
All 20 master decisions, all 6 stack additions, and all 5 phase confirmations are LOCKED prior to construction. No retroactive changes without an explicit revision and a new version of this document.

Part II — Architecture & People

3. The 9-Module Architecture Map

CMCMIS_SIMPLIFIED is organised around nine logical modules. Six are in the 10-week MVP; three (Schedule, Procurement, Reports) plus the Master Data Management UI are deferred to Phase 2. The module map below shows ownership of data and the natural read/write flow between modules.

3.1 The Big Picture



3.2 Module Responsibility Matrix

#	Module	Owns (CRUD)	Primary Actors	MVP?
1	Dashboard	Nothing (read-only)	All roles	YES
2	Job Requests	job_requests	Normal User → Lab In-Charge	YES
3	Job Cards	job_cards, tasks, observations	Lab Engineer, Lab In-Charge	YES
4	Equipment	equipment, specs, cal_history	All except View-Only (4 roles can create)	YES
5	Schedule	schedules (PM + Cal)	Lab In-Charge, Engineer	Phase 2

#	Module	Owns (CRUD)	Primary Actors	MVP?
6	Procurement	purchase_orders, spare_parts	Lab In-Charge, Super Admin	Phase 2
7	Vendors	vendors	Super Admin	Phase 2
8	Inquiry	Nothing (read-only)	All roles	YES
9	Reports	Nothing (read-only)	Lab In-Charge, Super Admin	Phase 2
10	Admin	Master data + Users + Roles	Super Admin only	Phase 2 (RBAC actions in MVP)

3.3 The Golden Rule of Data Flow

Inter-module writes flow in exactly one direction. This is the single most important architectural invariant in the system.

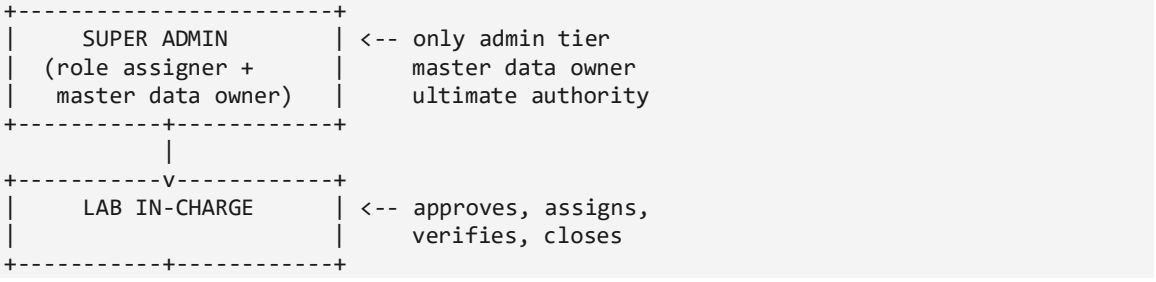
```
EQUIPMENT (source of truth)
       |
       v
JOB REQUEST (references equipment_id)
       |
       v
JOB CARD (references job_request_id + equipment_id)
       |
       v
DOCUMENT (references job_card_id, generated on demand)
       |
       v
EQUIPMENT.last_calibration_date  <-- the ONLY back-write
(and it goes through the state machine)
```

> **⚠ IMPORTANT**
> If Job Cards could freely modify Equipment, the audit trail breaks. ONE well-defined hook back-writes calibration metadata — through the state machine, never directly. This rule is enforced by code review and by middleware.

4. Role Model (5 Roles, Single-Tier Admin)

CMCMIS_SIMPLIFIED operates on a 5-role hierarchy. The earlier draft included a separate 'Admin' role; this has been collapsed into Super Admin to eliminate role-overlap ambiguity. Super Admin is now the single administrative tier and absorbs all master data management and system configuration responsibilities.

4.1 Role Hierarchy Diagram



```
                    |
             +------v------+
             |  LAB ENGINEER  | <-- executes work,
             |                |     fills job cards
             +------+------+
                    |
            +-------+-------+
            v               v
  +-------------------+   +-------------------+
  |   NORMAL USER     |   |   VIEW-ONLY USER  |
  | raises requests + |   | read-only — no    |
  | registers equip!  |   | writes EVER       |
  +-------------------+   +-------------------+
```

4.2 Role Responsibilities At a Glance

Role	Primary Responsibility	Bootstrap
Super Admin	Role assignment, master data management, system integrity, audit oversight	Seeded via env CSV (≥2 employee IDs)
Lab In-Charge	Approve job requests, assign engineers, verify completed work, manage schedules	Assigned by Super Admin
Lab Engineer	Execute assigned jobs, fill job cards, record observations, generate PDFs	Assigned by Super Admin
Normal User	Raise job requests, register equipment, track own requests	Default for any new login without role
View-Only User	Read all data; no write actions ever; auditor / management oversight	Assigned by Super Admin

4.3 Equipment Registration — Cross-Role Capability (NEW BEHAVIOUR)

All roles except View-Only can register a new equipment. Even a Normal User can register. Verification (PENDING_VERIFICATION → ACTIVE) still requires Lab In-Charge or Super Admin.

> ⚠ **IMPORTANT**
> Concentrating registration permission across four roles is unusual for metrology. The system mitigates the risk by (a) starting every new equipment in PENDING_VERIFICATION, (b) recording registered_by + timestamp on every record, and (c) requiring Lab In-Charge or Super Admin to verify before the equipment becomes operational.

5. Authorization & RBAC (3-Layer Model)

Permissions in CMCMIS_SIMPLIFIED are implemented as a three-layer model. Two-layer RBAC (User → Role) requires a redeploy every time business rules change about who can do what. Three-layer RBAC (User → Role → Permission → Resource:Action) lets Super Admin toggle a checkbox and have behaviour change instantly — no redeploy.

5.1 The Three Layers

```
    USER ---> ROLE ---> PERMISSION ---> RESOURCE+ACTION
      |          |            |                |
```

```
|          |           |                +-- "equipment:create"
|          |           +-- Granular toggle stored in DB
|          +-- Role bundle (e.g. "Lab Engineer")
+-- Identity (employee_id)
```

Layer	Real-World Example	Stored In
User	Employee #EMP1234 (Deep)	users
Role	Lab Engineer	roles + user_roles
Permission	job_card:update, equipment:read	permissions + role_permissions
Resource : Action	Checked at API middleware (runtime)	Runtime check

5.2 The Code Contract — BR-RBAC-03

✓ **LOCKED**
Code never checks by role name. Code always checks by permission. This is the architectural contract. A controller asks 'does this user have job_card:verify-close?' — it does NOT ask 'is this user a Lab In-Charge?'. Business rule changes happen in the database, not in the codebase.

6. Final Consolidated Permission Matrix

This is the authoritative reference. Every API endpoint and UI affordance is gated by exactly one permission from this matrix. The codebase reads this matrix from the database at startup; it is not hard-coded in source files.

6.1 Authentication & Identity

Resource : Action	[illegible]	View-Only	Lab Engineer	Lab In-Charge	Super Admin
auth:login	✓	✓	✓	✓	✓
auth:logout	✓	✓	✓	✓	✓
auth:refresh-token	✓	✓	✓	✓	✓
me:read (own profile)	✓	✓	✓	✓	✓

6.2 User & Role Management

Resource : Action	No[illegible]	View-Only	Lab Engineer	Lab In-Charge	Super Admin
user:read-list	✗	✗	✗	✗	✓
user:role-assign	✗	✗	✗	✗	✓
user:activate / deactivate	✗	✗	✗	✗	✓

6.3 Equipment

Resource : Action	Normal	View-Only	Lab Engineer	Lab In-Charge	Super Admin
equipment:read-list	✓	✓	✓	✓	✓
equipment:read-detail	✓	✓	✓	✓	✓
equipment:create (NEW)	✓	✗	✓	✓	✓
equipment:update	✗	✗	✓	✓	✓
equipment:verify (PENDING → ACTIVE)	✗	✗	✗	✓	✓
equipment:condemn (status flip)	✗	✗	✗	✓	✓
equipment:delete (hard)	✗	✗	✗	✗	✓

6.4 Job Requests

Resource : Action	Normal	View-Only	Lab Engineer	Lab In-Charge	Super Admin
job_request:create	✓	✗	✓	✓	✓
job_request:read-own	✓	✓	✓	✓	✓
job_request:read-all	✗	✓	✓	✓	✓
job_request:approve	✗	✗	✗	✓	✓
job_request:reject	✗	✗	✗	✓	✓
job_request:assign-engineer	✗	✗	✗	✓	✓

6.5 Job Cards

Resource : Action	Normal	View-Only	Lab Engineer	Lab In-Charge	Super Admin
job_card:read-list	✗	✓	✓	✓	✓
job_card:read-detail	✗	✓	✓	✓	✓
job_card:start-work (own)	✗	✗	✓	✓	✓
job_card:update-tasks (own)	✗	✗	✓	✓	✓
job_card:complete (own)	✗	✗	✓	✓	✓
job_card:verify-close	✗	✗	✗	✓	✓
job_card:reopen	✗	✗	✗	✓	✓
job_card:generate-pdf	✗	✓	✓	✓	✓

6.6 Dashboard & Inquiry

Resource : Action	Normal	View-Only	Lab Engineer	Lab In-Charge	Super Admin
dashboard:view	✓	✓	✓	✓	✓
inquiry:search-vendors	✓	✓	✓	✓	✓
inquiry:search-products	✓	✓	✓	✓	✓
inquiry:search-job-cards	✗	✓	✓	✓	✓
inquiry:search-instruments	✓	✓	✓	✓	✓

6.7 Master Data (Phase 2, Super Admin only)

Resource : Action	Normal	View-Only	Lab Engineer	Lab In-Charge	Super Admin
master:employees:manage	✗	✗	✗	✗	✓
master:vendors:manage	✗	✗	✗	✗	✓
master:equipment-types:manage	✗	✗	✗	✗	✓
master:divisions:manage	✗	✗	✗	✗	✓
master:lookup-values:manage	✗	✗	✗	✗	✓

6.8 Audit & Logs

Resource : Action	Normal	View-Only	Lab Engineer	Lab In-Charge	Super Admin
audit_log:read	✗	✗	✗	✗	✓
export:trigger (PDF / future Excel)	✗	✗	✓	✓	✓

INFO

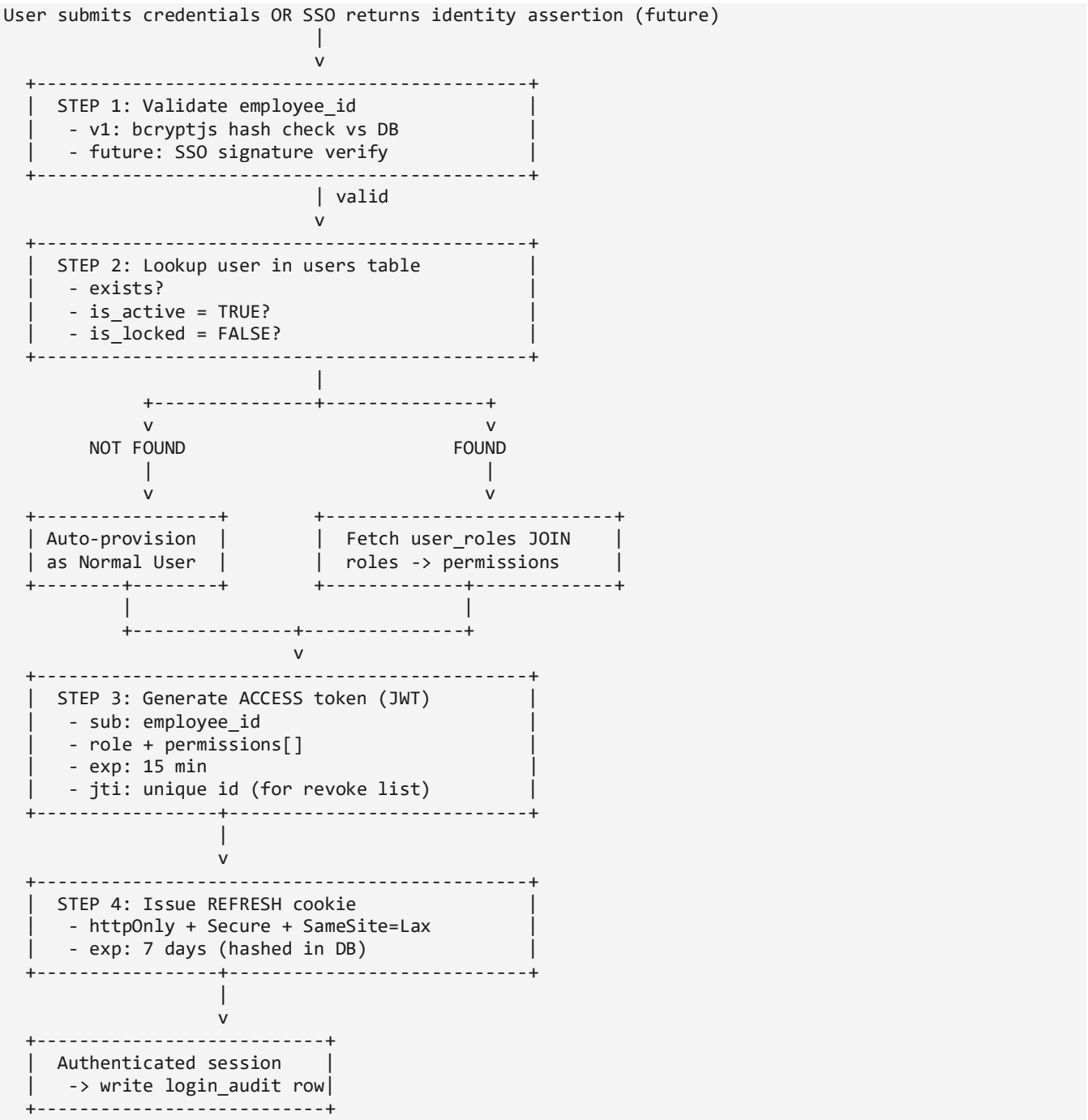
Row-level visibility applies in addition to the matrix: Normal User sees only own job requests; Lab Engineer sees own assignments + the team queue; Lab In-Charge / Super Admin / View-Only see all records.

Part III — Authentication, State Machines, Rules

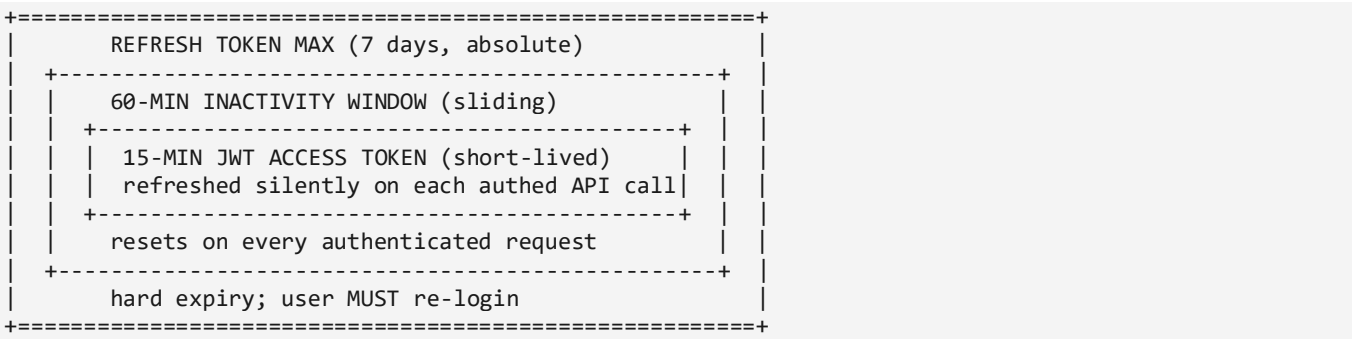
7. Authentication Flow

Authentication in v1 is by employee_id + password against the existing organisational database. The system is architected as SSO-ready, but does not depend on SSO. When the organisation later activates Active Directory, the authentication adapter is swapped — the rest of the system is unaffected.

7.1 Login Flow (Step-by-Step)



7.2 Session Envelopes (Three Concentric Timers)



Three timers operate concurrently. The 15-minute JWT is silently refreshed on every authenticated request. The 60-minute inactivity window resets on every authenticated request — if the user is idle for 60 minutes, the session expires. The 7-day refresh token is an absolute ceiling — even an active user must re-login after 7 days.

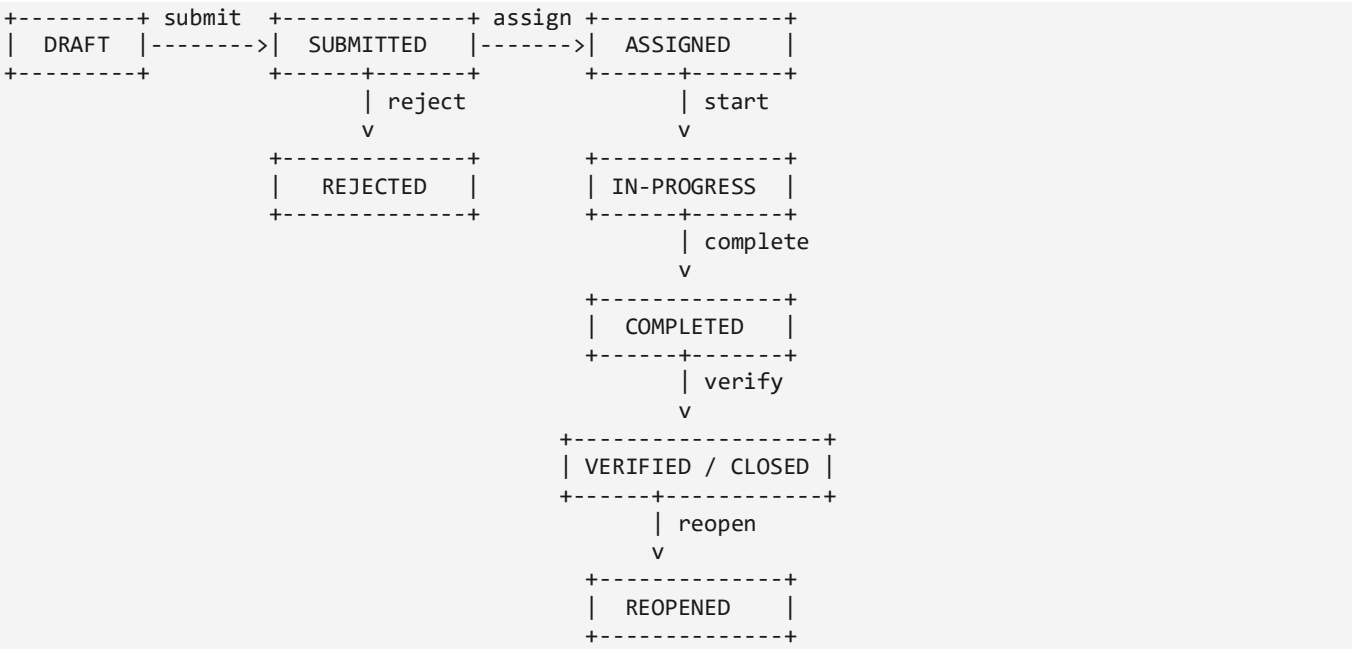
7.3 First Super Admin Bootstrap

- Seeded via DB migration on first deploy (not manual SQL)
- Env var: SUPER_ADMIN_EMPLOYEE_IDS (comma-separated, at least 2 employee IDs)
- Migration inserts: users + user_roles (SUPER_ADMIN) + audit_log 'BOOTSTRAP' rows
- First login of each seeded Super Admin forces password reset (password_must_change = TRUE)

8. Critical State Machines

Three explicit state machines govern the behaviour of CMCMIS_SIMPLIFIED. Defining states and allowed transitions formally — rather than implicitly through scattered if-statements — prevents data corruption bugs and makes the system auditable.

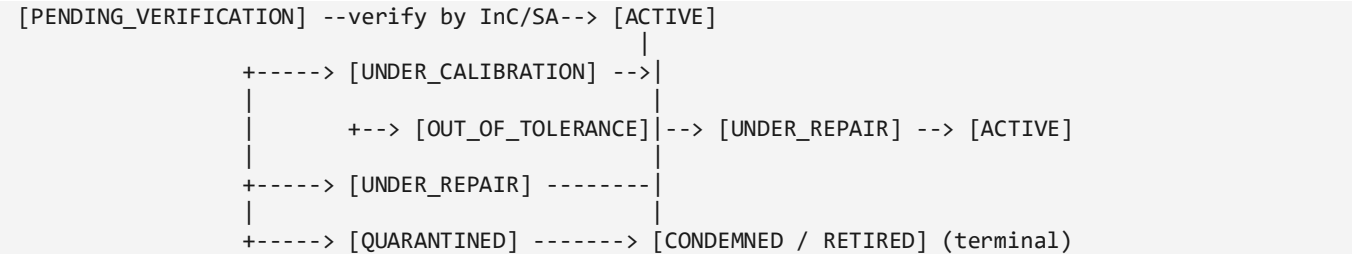
8.1 Job Request Lifecycle



From → To	Allowed By	Side Effects
DRAFT → SUBMITTED	Owner	Notify Lab In-Charge
SUBMITTED → ASSIGNED	Lab In-Charge	Auto-create Job Card
SUBMITTED → REJECTED	Lab In-Charge	Mandatory reason; lock further edits
ASSIGNED → IN-PROGRESS	Assigned Engineer	Set start_time
IN-PROGRESS → COMPLETED	Assigned Engineer	Required: observations exist (BR-JC-07)
COMPLETED → VERIFIED/CLOSED	Lab In-Charge	Generate cert; update Equipment.last_cal_date
VERIFIED → REOPENED	Lab In-Charge / Super Admin	Audit log with mandatory reason

8.2 Equipment Lifecycle (with PENDING_VERIFICATION)

All equipment registrations start at PENDING_VERIFICATION. Only Lab In-Charge or Super Admin can flip the status to ACTIVE.



8.3 Implementation Pattern — Single Choke-Point

Every state transition flows through one function per entity. No raw UPDATE statements modify status anywhere else in the codebase. Each transition writes to a *_status_history table automatically. Audit trail comes for free.

```
// One function per entity. Single source of truth.
function transitionJobRequest(currentState, action, actor) {
  const allowed = ALLOWED_TRANSITIONS[currentState]?.[action];
  if (!allowed) throw new Error('Illegal transition');
  if (!actor.hasPermission(allowed.permission)) throw new Error('Forbidden');
  return { newState: allowed.to, sideEffects: allowed.effects };
}
```

8.4 Calibration Status (Display Badges)

Status	Meaning	Color
VALID	Within calibration period	Green
DUE_SOON	Less than 30 days to next calibration	Yellow
OVERDUE	Past calibration due date	Red
OUT_OF_TOLERANCE	Last calibration failed	Black

Status	Meaning	Color
NOT_REQUIRED	F&PE equipment, calibration not applicable	Grey

9. Business Rules Catalogue

Business rules are the contract between the business and the codebase. Each rule has a unique identifier and is enforced either at the API layer, the database layer, or both. 36+ rules across 9 categories.

9.1 BR-AUTH — Authentication & Identity

ID	Rule
BR-AUTH-01	Login is by employee_id only (not email / username)
BR-AUTH-02	User must exist in org's employee directory — no self-registration
BR-AUTH-03	Authenticated user with no role → defaults to Normal User
BR-AUTH-04	Sessions expire after 60 minutes of inactivity. Refresh token valid 7 days.
BR-AUTH-05	First Super Admin seeded via DB migration using SUPER_ADMIN_EMPLOYEE_IDS env var. Seed at least 2.
BR-AUTH-06	All login attempts (success + failure) are logged
BR-AUTH-07	Deactivated user (status = INACTIVE) cannot log in but history is preserved

9.2 BR-RBAC — Authorization

ID	Rule
BR-RBAC-01	Only Super Admin can assign / change a user's role
BR-RBAC-02	A user has exactly ONE primary role at a time (no multi-role in v1)
BR-RBAC-03	Permissions derived: User → Role → Permissions. NEVER check by role name.
BR-RBAC-04	Sidebar items and routes are filtered by permissions, not role names
BR-RBAC-05	Every API endpoint enforces permission check at the controller layer
BR-RBAC-06	View-Only users can read but cannot trigger any write operation
BR-RBAC-07	Role changes take effect on next login OR token refresh

9.3 BR-EQP — Equipment

ID	Rule
BR-EQP-01	Every equipment has a unique serial number system-wide

ID	Rule
BR-EQP-02	Equipment must belong to T&ME or F&PE category
BR-EQP-03	F&PE may have NO calibration frequency (optional)
BR-EQP-04	T&ME MUST have a calibration frequency (months)
BR-EQP-05	next_cal_due_date = last_cal_date + calibration_frequency_months
BR-EQP-06	Status transitions follow the equipment state machine
BR-EQP-07	Hard DELETE = Super Admin only; CONDEMN = Lab In-Charge or Super Admin
BR-EQP-08	Search is case-insensitive across serial, model, manufacturer, type
BR-EQP-09	Every registration captures registered_by + verified_by + timestamps
BR-EQP-10	New equipment defaults to PENDING_VERIFICATION; verified by Lab In-Charge / Super Admin → ACTIVE

9.4 BR-JR — Job Requests

ID	Rule
BR-JR-01	Must reference existing equipment (or trigger registration)
BR-JR-02	Job type ∈ {Calibration, Repair, Registration}
BR-JR-03	User can save as DRAFT before submitting
BR-JR-04	Once SUBMITTED, only Lab In-Charge can change state
BR-JR-05	Submitted request must be assigned to a Lab Engineer before becoming a Job Card
BR-JR-06	'submitted by' auto-filled from current user — not overridable
BR-JR-07	High-priority repairs appear at top of Lab In-Charge queue
BR-JR-08	Rejection requires mandatory reason (free text + reason code)

9.5 BR-JC — Job Cards

ID	Rule
BR-JC-01	Auto-created when a request is approved + assigned
BR-JC-02	Lifecycle: ASSIGNED → IN-PROGRESS → COMPLETED → VERIFIED/CLOSED
BR-JC-03	Only the assigned engineer can mark IN-PROGRESS or COMPLETED
BR-JC-04	Only Lab In-Charge can verify / close
BR-JC-05	Closed job can only be REOPENED by Lab In-Charge with reason

ID	Rule
BR-JC-06	Tasks are configurable per job type
BR-JC-07	Calibration cards require before-reading + after-reading + environment before COMPLETED
BR-JC-08	Job card history is append-only — immutable state-transition log

9.6 BR-PDF, BR-AUD, BR-VIS, BR-MASTER

ID	Rule
BR-PDF-01	PDFs generated on demand from current DB state — nothing stored
BR-PDF-02	Job card PDF: header, equipment info, tasks, observations, signatures, date
BR-PDF-03	Calibration cert PDF: equipment, standards, readings, environment, uncertainty, valid-until
BR-PDF-04	All PDFs include generation timestamp + footer + record ID
BR-AUD-01	Every write on critical tables is logged in audit_logs
BR-AUD-02	Audit log captures who, what, when, before/after, IP, user-agent
BR-AUD-03	Exports logged with user + record IDs accessed
BR-AUD-04	Sensitive fields filtered from API responses based on permission
BR-AUD-05	HTTPS in prod. JWT in Authorization header. Refresh in httpOnly cookie.
BR-VIS-01	Normal User sees only their own job requests
BR-VIS-02	Lab Engineer sees all jobs assigned to them + the queue
BR-VIS-03	Lab In-Charge and above see all jobs and equipment
BR-VIS-04	View-Only sees all data, performs no actions
BR-MASTER-01	All master-data CRUD is Super Admin only

Part IV — Requirements

10. Functional Requirements (MVP)

45 functional requirements across six MVP modules. 44 are MUST priority; 1 is SHOULD. Each FR has a unique identifier and traces back to one or more business rules.

10.1 FR-A — Auth & RBAC Module

FR ID	Description	Priority
FR-A-01	Login screen accepts employee_id + password	MUST
FR-A-02	Successful login returns JWT + sets refresh cookie	MUST
FR-A-03	Logout clears tokens and invalidates session	MUST
FR-A-04	Protected routes redirect to login when unauthenticated	MUST
FR-A-05	Sidebar items render based on user's permission set	MUST
FR-A-06	Super Admin can view list of all users + their roles	MUST
FR-A-07	Super Admin can assign / change a user's role	MUST
FR-A-08	Super Admin can activate / deactivate a user	MUST
FR-A-09	Backend exposes GET /me returning current user + permissions	MUST
FR-A-10	Token refresh endpoint silently extends session	MUST

10.2 FR-E — Equipment Module

FR ID	Description	Priority
FR-E-01	List equipment with pagination, search, filter (type, status)	MUST
FR-E-02	View equipment detail page with specs, cal history, linked jobs	MUST
FR-E-03	Register new equipment (available to all roles except View-Only)	MUST
FR-E-04	Edit equipment details (specs, status)	MUST
FR-E-05	Color-code calibration due dates (green / yellow / red)	MUST
FR-E-06	Show equipment's calibration history timeline	MUST
FR-E-07	Search equipment by serial, model, manufacturer	MUST
FR-E-08	Soft-delete (status = CONDEMNED) for non-Super Admins	MUST
FR-E-09	Lab In-Charge / Super Admin can verify equipment (PENDING → ACTIVE)	MUST

10.3 FR-JR — Job Request Module

FR ID	Description	Priority
FR-JR-01	List job requests with filters (type, status, my requests vs all)	MUST
FR-JR-02	Create job request form (multi-section, progressive disclosure)	MUST
FR-JR-03	Save as draft	MUST
FR-JR-04	Submit triggers state transition to SUBMITTED	MUST
FR-JR-05	View request detail with full info + state history	MUST
FR-JR-06	Lab In-Charge can approve / reject with reason	MUST
FR-JR-07	Approval triggers automatic Job Card creation	MUST
FR-JR-08	Assignment dropdown shows lab engineers with current workload	SHOULD

10.4 FR-JC — Job Card Module

FR ID	Description	Priority
FR-JC-01	List job cards with filter (status, assigned-to-me, all)	MUST
FR-JC-02	View job card with horizontal status stepper	MUST
FR-JC-03	Task checklist (configurable per job type)	MUST
FR-JC-04	Observations & readings log (free text + structured)	MUST
FR-JC-05	Engineer can transition state (Start work, Mark complete)	MUST
FR-JC-06	Lab In-Charge can Verify / Close or Reopen	MUST
FR-JC-07	Generate PDF of job card (on-demand download)	MUST
FR-JC-08	State history visible as timeline	MUST

10.5 FR-D — Dashboard & FR-I — Inquiry

FR ID	Description	Priority
FR-D-01	Role-aware KPI widgets (different per role)	MUST
FR-D-02	Calibration due alerts (next 30 days + overdue)	MUST
FR-D-03	Engineer workload bar chart (for Lab In-Charge)	MUST
FR-D-04	Equipment status pie chart	MUST
FR-D-05	Recently updated jobs table	MUST
FR-D-06	Quick actions: Raise request, Add equipment (permission-gated)	MUST
FR-I-01	4-tab interface (Vendor, Product, Job Card, Instrument)	MUST

FR ID	Description	Priority
FR-I-02	Real-time search with debounce	MUST
FR-I-03	Result tables with drill-down link to full record	MUST
FR-I-04	Empty-state messages when no results	MUST

11. Non-Functional Requirements

Non-functional requirements constrain how the system behaves rather than what it does. They are equally important and equally testable. These targets are the v1 commitment.

Category	Requirement	Target
Performance	Initial page load (cold, on intranet)	< 3 seconds
Performance	API response (95th percentile)	< 500 ms
Performance	List pagination	25 items default, 100 maximum
Security	Password hashing	bcrypt, ≥ 10 rounds (12 in prod)
Security	JWT access lifetime	15 minutes
Security	Refresh token lifetime	7 days, httpOnly cookie, SameSite=Lax
Security	SQL injection defence	Parameterised queries everywhere
Security	XSS	React auto-escape + CSP header
Security	CSRF	SameSite cookies + CSRF token on /auth/refresh
Reliability	Server uptime (post go-live)	99% during business hours
Reliability	DB connection pool	10–20 connections
Usability	Max clicks to any feature	≤ 3
Usability	Keyboard shortcuts on power screens	Available
Usability	Responsive viewport	1280–1920 primary; graceful to 768
Audit	Every state-changing operation logged	100% coverage
Audit	Audit log retention	Indefinite (until specified)
Maintainability	Code style	ESLint + Prettier (enforced via husky)
Maintainability	Folder structure	Strict layered separation
Maintainability	API versioning	/api/v1/... from day one

Category	Requirement	Target
Compatibility	Browsers	Chrome / Edge / Firefox (latest 2 versions)

Part V — Tech Stack & System Architecture

12. Tech Stack (Final & Locked)

Every library has been selected against three criteria: maturity and community support, minimal surface area to learn for a solo developer, and good fit for the project constraints (no Redis, no email, no file storage, MySQL only, on-prem deployment).

12.1 Frontend Stack (React 18 + Vite + Tailwind v3)

#	Concern	Library	Why
1	Build	vite	Fast HMR, modern default
2	Framework	react@18	Locked
3	Routing	react-router-dom v6	SPA standard
4	State (local)	useState / useReducer	Built-in
5	State (global UI)	zustand	~1 KB, no boilerplate
6	Server state	@tanstack/react-query	Cache / refetch / mutations
7	HTTP	axios	Interceptors for auth / refresh
8	Forms	react-hook-form + zod	Performant + schema shared with BE
9	UI primitives	Custom on Tailwind	Avoids shadcn / MUI lock-in
10	Icons	lucide-react	Tree-shakable
11	Charts	recharts	Dashboard widgets
12	Date	dayjs	Symmetric with backend
13	Toasts	sonner	Lightweight
14	Styling	tailwindcss v3	Locked
15	Table primitive	@tanstack/react-table	Headless table for equipment / job lists (S4)
16	Form defaults	@tailwindcss/forms	Sane form base layer (S5)
17	Lint / Format	eslint + prettier + husky + lint-staged	Pre-commit enforcement

12.2 Backend Stack (Node + Express 4)

#	Concern	Library	Ver	Why
1	Web framework	express	^4.x	Stable, ubiquitous
2	DB driver	mysql2/promise	^3.x	Fastest MySQL driver

#	Concern	Library	Ver	Why
3	Validation	zod	^3.x	Single source of truth (FE+BE share)
4	Auth tokens	jsonwebtoken	^9.x	Industry standard
5	Password hash	bcryptjs	^2.x	Pure-JS, no native build pain
6	Logger	pino + pino-pretty	^8.x	Faster than Winston, JSON
7	Env loader	dotenv	^16.x	Standard
8	Env validation	envalid	^8.x	Fail at boot if env missing
9	Date	dayjs	^1.x	Symmetric with frontend
10	PDF	pdfkit	^0.14.x	Programmatic, no Chromium
11	Rate limit	express-rate-limit	^7.x	In-memory, sufficient v1
12	Security	helmet	^7.x	OWASP-friendly defaults
13	CORS	cors	^2.x	Standard
14	Process mgr	pm2	latest	Cluster mode, restarts, logs
15	Cookie parsing	cookie-parser	latest	Read httpOnly refresh cookie (S1)
16	Compression	compression	latest	gzip / brotli — supports p95 target (S2)
17	CSRF defence	custom double-submit	—	CSRF token on /auth/refresh (S3)
18	Testing (unit)	vitest	latest	Auth + state machine unit tests (S6)
19	Testing (API)	supertest	latest	API integration tests (S6)
20	Lint / Format	eslint + prettier + husky	latest	Pre-commit

12.3 The Three Big Architectural Wins

- Schema symmetry — Zod on BOTH frontend and backend → one definition, two enforcers. The same schema validates the form in the browser and the request body on the server.
- Date symmetry — dayjs on BOTH sides → no timezone or format drift on critical math like BR-EQP-05 (next_cal_due_date = last_cal_date + frequency).
- On-prem friendly — pdfkit (no Chromium), pm2 (no systemd quirks), no Redis / SMTP / object-storage dependencies. The entire stack can be installed offline from a local package mirror.

12.4 What We Deliberately Did Not Pick

Library / Tool	Reason for Rejection
Redux / Redux Toolkit	Overkill for this scale; Zustand + React Query covers it

Library / Tool	Reason for Rejection
Sequelize / Prisma / TypeORM	Existing 64-table DB + metrology complexity = raw SQL wins
GraphQL	Adds a layer with zero v1 benefit; REST is correct
TypeScript (v1)	JS + JSDoc + Zod is faster to ship; can migrate later
Material-UI / Ant Design	Heavy, opinionated, hard to escape
Next.js	SSR irrelevant for internal SPA; Vite is faster to develop in
MongoDB / Mongoose	MySQL was chosen by the organisation
Redis	MySQL + pagination + connection pool is sufficient at expected scale
SMTP / email	Out of scope; will be added when org provides mail relay
S3 / object storage	PDFs generated on demand; no files persisted

13. System Architecture & Request Trace

13.1 Runtime Architecture

```
+-------------------------------------------------------------+
| DESKTOP / LAPTOP BROWSER  (NO MOBILE)                       |
|   React 18 + Vite + Tailwind                                |
|   Routes        -> React Router                             |
|   Server state  -> TanStack Query                           |
|   Forms         -> React Hook Form + zod                    |
|   HTTP          -> axios (interceptor attaches JWT)         |
+--------------------------+----------------------------------+
                           | HTTPS  (JWT + httpOnly refresh)
                           v
+-------------------------------------------------------------+
| NODE + EXPRESS API SERVER  (on-prem)                        |
|   Middleware:  helmet > cors > compression > json > cookies |
|                > pino > rate-limit > auth > authorize > scope |
|                > validate(zod) > controller                 |
|   Service layer:  state-machine transition()                |
|                   pdfkit cert generator                     |
|                   audit-log helper                          |
|   Data access:    mysql2/promise pool, parameterised SQL    |
+--------------------------+----------------------------------+
                           v
+-------------------------------------------------------------+
| MySQL 8.x  (existing on-prem; phpMyAdmin admin)             |
|   ~70 tables (existing 64 + new) · InnoDB · utf8mb4         |
|   FK + CHECK constraints · indexes per dashboard / inquiry  |
+-------------------------------------------------------------+

NOT IN MVP:  Redis · SMTP · S3 / object store · SSO / AD
```

13.2 End-to-End Request Trace (Equipment Register)

Following a single user action — a Lab Engineer registering a new equipment — through every layer of the system.

```
Lab Engineer fills equipment register form
```

```
 -> react-hook-form + zod validates on submit (no network if invalid)
 -> @tanstack/react-query mutation calls axios.post('/api/v1/equipment')
 -> axios interceptor attaches Bearer JWT
----------------- NETWORK BOUNDARY -----------------
 -> helmet (headers) > cors > compression > express.json > cookie-parser
 -> pino logs request meta
 -> express-rate-limit (auth routes only)
 -> authenticate (jsonwebtoken verify)
 -> authorize ('equipment:create' permission check)
 -> rowLevelScope (no-op for create)
 -> validate(zod) re-checks body
 -> equipmentController.create
 -> equipmentService.create:
       - dayjs.add for next_cal_due_date
       - state machine: status = PENDING_VERIFICATION
       - audit_log row
 -> mysql2 transaction: INSERT equipment + INSERT audit_log
 -> 201 Created + JSON body
 -> pino logs response status + duration
----------------- NETWORK BOUNDARY -----------------
 -> react-query invalidates 'equipment-list' cache
 -> sonner: 'Equipment registered. Pending verification.'
 -> router navigates to detail page
```

14. Express Middleware Pipeline

Every API request passes through the same middleware chain in the same order. Order matters: security headers come before parsing; authentication comes before authorization; validation comes before the controller; the central error handler always closes the chain.

```
     incoming request
            |
            v
+-----------------------------------------------------+
|  1. helmet()                <- security headers     |
|  2. cors(orgAllowlist)      <- reject foreign origs |
|  3. compression()           <- gzip responses       |
|  4. express.json({limit})   <- parse JSON body      |
|  5. cookie-parser()         <- parse refresh cookie |
|  6. pino-http()             <- log request meta     |
|  7. rateLimit(login)        <- only on auth routes  |
|  8. authenticate()          <- JWT verify           |
|  9. authorize(perm)         <- RBAC check           |
| 10. rowLevelScope()         <- BR-VIS WHERE clause  |
| 11. validate(zodSchema)     <- input validation     |
| 12. controller              <- business handler     |
| 13. errorHandler()          <- centralised errors   |
+-----------------------------------------------------+
            |
            v
     outgoing response
```

#	Middleware	Purpose
1	helmet	OWASP-friendly security headers (HSTS, CSP, XFO, etc.)
2	cors	Reject requests from origins not in the org allow-list
3	compression	gzip / brotli responses for bandwidth + p95 latency
4	express.json	Parse JSON request bodies with a size limit

#	Middleware	Purpose
5	cookie-parser	Parse the httpOnly refresh-token cookie
6	pino-http	Structured request / response logging with requestId
7	express-rate-limit	Throttle abuse on auth endpoints
8	authenticate	Verify JWT signature + expiry; attach req.user
9	authorize(permission)	Check the required resource:action permission
10	rowLevelScope	Apply BR-VIS WHERE clauses (e.g. own jobs for Normal User)
11	validate(zodSchema)	Re-validate body / query / params against schema
12	controller	Business handler — thin, delegates to service
13	errorHandler	Convert thrown errors into standard error envelope

Part VI — Structure, Conventions, Scope, Roadmap

15. Project Folder Structure

CMCMIS_SIMPLIFIED is a mono-repo with two independent projects: backend (Node.js + Express) and frontend (React + Vite). They share zod schemas via a common path so that input shape is the same on both sides of the network boundary.

15.1 Top-Level Structure

```
cmcmis-simplified/
 |
 +-- backend/
 |    +-- src/
 |    |    +-- config/          (db.js, env.js, logger.js, jwt.js)
 |    |    +-- middleware/      (authenticate, authorize, rowLevelScope,
 |    |    |                     validate, errorHandler)
 |    |    +-- modules/
 |    |    |    +-- auth/        (routes, controller, service, schema)
 |    |    |    +-- users/
 |    |    |    +-- equipment/
 |    |    |    +-- jobRequests/
 |    |    |    +-- jobCards/
 |    |    |    +-- dashboard/
 |    |    |    +-- inquiry/
 |    |    |    +-- audit/
 |    |    +-- stateMachines/   (jobRequest.fsm.js, equipment.fsm.js)
 |    |    +-- pdf/             (jobCard.pdf.js, calibrationCert.pdf.js)
 |    |    +-- utils/
 |    |    +-- server.js
 |    +-- db/
 |    |    +-- migrations/      (numbered .sql files)
 |    |    +-- seeds/           (roles, permissions, super admins)
 |    |    +-- schema.sql
 |    +-- tests/
 |    +-- .env.example
 |    +-- ecosystem.config.js   (pm2)
 |    +-- package.json
 |
 +-- frontend/
 |    +-- src/
 |    |    +-- api/             (axios client + per-module hooks)
 |    |    +-- components/      (Button, Input, Table, Modal, FormField)
 |    |    +-- features/        (auth, equipment, jobRequests, jobCards,
 |    |    |                     dashboard, inquiry)
 |    |    +-- layouts/         (AppLayout, AuthLayout)
 |    |    +-- lib/             (authContext, permissions, formatters)
 |    |    +-- pages/           (thin route components)
 |    |    +-- schemas/         (zod schemas — shared with BE)
 |    |    +-- stores/          (zustand)
 |    |    +-- router.jsx
 |    |    +-- main.jsx
 |    +-- public/
 |    +-- tailwind.config.js
 |    +-- vite.config.js
 |    +-- package.json
 |
 +-- docs/
 |    +-- ERD.md
 |    +-- rbac-matrix.md
 |    +-- state-machines.md
 |    +-- business-rules.md
```

```
+-- .gitignore
+-- .editorconfig
+-- README.md
+-- package.json   (monorepo root)
```

15.2 Backend Layered Flow

```
HTTP request
     |
     v
 +------------+
 |   ROUTE    |  <- URL pattern + middleware chain
 +-----+------+
       v
 +------------+
 | CONTROLLER |  <- THIN: parse req, call service, format response
 +-----+------+
       v
 +------------+
 |  SERVICE   |  <- THICK: business logic, state machines, transactions
 +-----+------+
       v
 +------------+
 | REPOSITORY |  <- raw SQL only, returns rows / domain objects
 +-----+------+
       v
 +------------+
 |   MySQL    |
 +------------+
```

Layer	Owns	Does NOT Own
Route	URL → handler mapping, middleware chain	Business logic
Controller	HTTP request / response shaping	SQL, business rules
Service	Business rules, transactions, state machines	HTTP concerns, raw SQL
Repository	SQL queries, parameter binding	Business logic, validation

16. Coding Conventions & API Standards

Conventions are enforced by tooling (ESLint, Prettier, husky pre-commit hooks) — not by documentation alone. New code that violates conventions cannot be committed.

Concern	Convention
API base	/api/v1/...
HTTP verbs	REST: GET / POST / PATCH / DELETE
Response shape (success)	{ data, meta }
Response shape (error)	{ error: { code, message, details } }
Status codes used	200 / 201 / 204 / 400 / 401 / 403 / 404 / 409 / 422 / 429 / 500
Pagination	?page=1&limit=25 (max 100)

Concern	Convention
Filtering	?status=ACTIVE&type=TME (whitelisted per endpoint)
Sorting	?sort=-created_at,name (- prefix = DESC)
Naming (DB)	snake_case, plural tables, id PKs, *_id FKs
Naming (JS)	camelCase vars / functions, PascalCase components / classes
Naming (files)	kebab-case.js or PascalCase.jsx for React components
Errors	Throw typed errors → centralised handler → standard response
Logs	Structured JSON via pino; never console.log
Commits	Conventional Commits (feat / fix / chore / ...)
Branches	feature/*, fix/*, chore/* from main

17. MVP Scope vs Phase 2 Backlog vs Out of Scope

17.1 MVP — 10 Weeks — Signed Off

- Auth + RBAC — login, session, role loading, permissions, protected routes, SSO-ready
- Equipment master + register + verify flow
- Job Requests — form, listing, full lifecycle
- Job Cards — tasks, observations, sign-off
- Dashboard — role-aware KPIs, alerts, widgets
- Inquiry — search hub (vendors, products, job cards, instruments)
- PDF generation — download / generate only — no storage
- Audit logs — basic, all state changes
- Responsive UI — desktop + laptop; 1280–1920 primary
- Row-level visibility — per BR-VIS rules
- Optimised SQL — pagination + connection pool

17.2 Phase 2 — Post-Internship Backlog

- Schedule module — Preventive Maintenance + Calibration calendar
- Procurement — Purchase Orders + spares inventory
- Reports — analytics + exports
- Admin Master Data CRUD UI
- Notifications — in-app feed

17.3 Explicitly NOT in MVP

Item	Decision
SSO / Active Directory integration	Architecture SSO-ready, integration deferred

Item	Decision
Email / SMTP	Out of scope until specified by org
Redis / caching layer	MySQL + pagination + pool is sufficient at expected scale
File storage	PDFs generated + downloaded only, never stored
Backup infrastructure	Out of scope (user handles separately)
Mobile / tablet UI	Desktop + laptop only (responsive within range)
Barcode / QR	Not in scope
NABL / ISO 17025 / AS9100 specifics	Deferred — user will instruct as needed
Calibration certificate template format	Deferred — user will instruct as needed
Notification channels (email / in-app / SMS / push)	Deferred — user will instruct as needed

18. 10-Week War Plan (Phased Timeline)

The week-by-week build sequence is the spine of the project. Each week has a single primary objective and a clear definition of done. Deviations from the weekly milestone are explicit and logged. The discipline rule: if a task is not on the line for the current week, it goes to the backlog. No exceptions.

Week	Phase	Deliverables
1	Phase 0 — Foundation	Repo scaffolding · ESLint / Prettier · Tailwind base layout · MySQL pool · 'hello world' protected route end-to-end
2	Phase 1 — DB Design	Inventory existing ~64 tables · Classify · Draft new tables (RBAC + Job + Equipment first) · Seed scripts · Lock ERD
3	Block 1 — Auth + RBAC	employee_id + password login · JWT + refresh · RBAC middleware (role + granular perm) · Sidebar visibility · Row-level scoping · Audit-log helper
4–5	Block 2a — Equipment Master	Equipment list / detail / register · State machine (incl. PENDING_VERIFICATION) · Cal history · Specs
5–6	Block 2b — Job Requests	Multi-section form (T&ME / F&PE; cal / repair / registration) · Draft / submit · Approve / reject · State machine
6–7	Block 2c — Job Cards + PDF	Auto-create on approval · Task checklist + observations · Verify / close → Equipment status update · PDF cert generation
8	Block 3 — Dashboard + Inquiry	Dashboard widgets (KPIs, due alerts, workload) · Inquiry 4-tab search
9	Hardening	Audit coverage check · RBAC e2e test (all 5 roles) · Illegal-transition tests · SQL tuning · Responsive QA

Week	Phase	Deliverables
10	Demo Prep + Deploy	Demo data seeded · On-prem deploy · Stakeholder demo · Bug-fix buffer

18.1 Slack / Buffer Strategy

Each week has a contingency plan if it falls behind. The principle: core (Auth, state machines, audit) is never cut. Surface features (dashboard widgets, inquiry tabs, PDF polish) are cut first.

Risk	Buffer Plan
Existing 64-table DB messier than expected	Phase 1 stretches into Week 3; Auth gets compressed
Job Card observations more complex than planned	Cut Inquiry tabs to 2 (job cards + equipment) for demo
PDF formatting takes longer	Ship plain-template PDFs; polish post-internship
RBAC edge cases discovered	Core — never cut. Cut Dashboard widgets instead.

19. Real-World End-to-End Walkthrough

Tracing a single instrument — Digital Multimeter DMM-2034 — through CMCMIS to make all the architecture concrete. This is the 'show-not-tell' validation that every module, role, state machine, and audit hook works together.

Step	Actor	Role	Module	Action
1	Engineer Deep	Lab Engineer	Dashboard	Sees DMM-2034 calibration due alert (yellow badge)
2	Deep	Lab Engineer	Job Requests	Raises calibration request for DMM-2034
3	Lab In-Charge	Lab In-Charge	Job Requests	Approves → Job Card auto-created and assigned
4	Deep (auto-assigned)	Lab Engineer	Job Cards	Completes tasks, enters readings, marks COMPLETE
5	Lab In-Charge	Lab In-Charge	Job Cards	Verifies + closes → triggers PDF cert generation
6	Anyone (except View-Only)	Any of 4 roles	Equipment	Could register new DMM-2035 tomorrow → PENDING_VERIFICATION
7	Lab In-Charge	Lab In-Charge	Equipment	Verifies DMM-2035 → ACTIVE
8	Super Admin	Super Admin	Admin (seed / CLI in MVP)	Promotes Normal User → Lab Engineer

Step	Actor	Role	Module	Action
9	View-Only auditor	View-Only	Inquiry	Searches DMM-2034 history; reads everything; edits nothing
10	Super Admin	Super Admin	Audit Log	Reviews every state transition for the calibration cycle

INFO

One instrument, one calibration cycle ≈ 20 database writes across ~12 tables, 1 PDF certificate, ~9 audit rows. This is why a ~70-table schema is the MINIMUM to track the lifecycle lawfully — anything less and the audit trail breaks.

Part VII — Database, Constraints, Next Step

20. Database Strategy & ~70-Table Map

CMCMIS_SIMPLIFIED does not start on a blank slate. Approximately 64 tables already exist with records in the organisation's database. Phase 3 of construction begins with a thorough audit of those tables — classifying each as 'definitely in use', 'probably dead', or 'unsure' — and reconciling against the target schema sketched below.

20.1 Reality Check

> ⚠ **IMPORTANT**
> Approximately 64 tables already exist with records. CMCMIS_SIMPLIFIED does NOT start on a blank slate — we refactor and extend the existing schema in Phase 3. Nothing existing is dropped in MVP; new tables are added; existing tables are extended only where necessary.

20.2 Target Entity Clusters (~70 Tables)

The data model is grouped into 13 entity clusters. Together these clusters total approximately 65–75 tables — well aligned with the existing ~64-table database. Exact reconciliation between target schema and existing tables is the work of Phase 3 (begins with input from DS).

#	Cluster	Approx. Tables	Example Entities
1	Identity & Access	8–10	users, roles, permissions, role_permissions, user_roles, sessions, login_audit
2	Organisation	4–5	divisions, departments, employees, designations, locations
3	Equipment Master	8–10	equipment, types, categories, specs, status, manufacturers, models, history
4	Job Lifecycle	6–8	job_requests, types, job_cards, tasks, observations, readings, state_history
5	Calibration	6–8	cal_records, standards, traceability, certs, env_conditions, uncertainty, OOT
6	Maintenance	4–5	maint_records, types, PM_plans, breakdowns, AMC
7	Scheduling (P2)	3–4	schedules, events, recurrence, notifications
8	Procurement (P2)	6–7	vendors, vendor_categories, POs, PO_lines, spares, stock, movements
9	Documents	3–4	docs, versions, types, signatures
10	Audit & Logs	3–4	audit_logs, change_history, login_audit, export_audit
11	Notifications (P2)	3	notifications, preferences, log
12	Lookups	5–7	units, statuses, frequencies, system_types

#	Cluster	Approx. Tables	Example Entities
13	Reporting (P2)	2–3	saved_reports, runs, scheduled
—	TOTAL	~65–75	Realistic for industry-grade metrology MIS

20.3 Phase 3 Approach (Starts with DS Inputs)

- Step 1 — DS delivers existing 64-table schema (SHOW CREATE TABLE outputs or mysqldump --no-data)
- Step 2 — DS delivers table classification: in-use / probably-dead / unsure
- Step 3 — DS delivers two Super Admin employee IDs for the bootstrap seed
- Step 4 — Existing schema audited cluster-by-cluster against the target 13 clusters above
- Step 5 — Reconciliation plan produced: keep / modify / drop / add for every table
- Step 6 — Final CREATE TABLE / ALTER TABLE scripts written and reviewed
- Step 7 — Seed scripts (roles, permissions, lookups, super admins) written and tested
- Step 8 — ER diagram drawn
- Step 9 — Migration runner set up (versioned SQL files, rollback procedure)

21. Locked Constraints (Quick Reference)

Every constraint below is closed. None is open to debate at this stage. If a constraint is later changed, a new revision of this document is issued with a new version number.

#	Constraint	Decision
1	Organisation context	ISRO SAC-like defence / space-grade environment
2	Existing DB	~64 tables; reviewed in Phase 3; never dropped in MVP
3	SSO in v1	NO — employee_id + password against existing DB; SSO-ready for future
4	Deployment target	Internal on-prem / org private infrastructure
5	File storage	NO — PDFs generated + downloaded only, never persisted
6	Email / SMTP	NO in v1
7	Redis / cache	NO — MySQL + pagination + connection pool only
8	Backup infrastructure	Out of scope (user handles separately)
9	Mobile / tablet UI	NO — desktop / laptop only; fully responsive within range
10	Barcode / QR	NO
11	PDF generation	YES — pdfkit, server-side, download only
12	Sensitive data	Defence-grade — row-level visibility + secure sessions + limited exports

#	Constraint	Decision
13	Concurrent user expectation	~50–200 (intranet user base)
14	Browser support	Chrome / Edge / Firefox latest 2 versions

22. Open Items — All Scheduled for Day 1 of Phase 3

Three pieces of input are required from DS to begin Phase 3. They are scheduled for delivery the morning Phase 3 begins.

#	Open Item	Owner	Delivery
O1	phpMyAdmin export of existing ~64 tables (SHOW CREATE TABLE or mysqldump --no-data)	DS	Day 1 morning of Phase 3
O2	Short note: which existing tables are definitely in-use / probably dead / unsure	DS	Day 1 morning of Phase 3
O3	Two Super Admin employee IDs for SUPER_ADMIN_EMPLOYEE_IDS env var	DS	Day 1 morning of Phase 3 (or during seeding in Week 2)

INFO

With these three inputs, Phase 3 can begin immediately. If the schema export takes time, Phase 3 can start by drafting the target schema (clean, ideal) and reconciling against existing tables once they arrive. Either order works.

23. Risk Register

Risks are tracked with severity, probability, and explicit mitigation. The register is reviewed weekly during the 10-week sprint and updated as risks emerge or close.

#	Risk	Severity	Probability	Mitigation
R1	Scope creep during internship	High	Very High	Strict weekly milestones, MVP discipline, weekly review
R2	Existing 64-table DB structure unknown	Medium	High	Phase 3 begins with full schema audit (Week 2)
R3	No backup infrastructure (user handles)	High	Certain	Logged; out of project scope; flagged for org
R4	No QA support (solo developer)	Medium	Certain	Weeks 8–9 reserved for self-QA + bug bash
R5	Stakeholder feedback delayed	Medium	High	Demo every 2 weeks if possible
R6	Solo developer burnout	High	Moderate	Sleep > rushed code; weekly milestones; rest discipline
R7	Compliance requirements surface mid-build	Medium	Moderate	Architecture stays compliance-friendly throughout

#	Risk	Severity	Probability	Mitigation
R8	Raw SQL slows initial development	Low	Moderate	Repository helpers + shared query utilities
R9	Audit logging adds latency	Low	Low	Async fire-and-forget write pattern
R10	Single Super Admin = bottleneck	Medium	High	Bootstrap requires ≥2 Super Admin IDs (D11)
R11	PDF generation complexity in week 6	Low	Moderate	Plain-template fallback; polish post-internship
R12	Existing DB has unknown FK constraints	Medium	Moderate	Schema audit identifies before any ALTER

24. Big Quick Recap (Print-and-Pin)

A single-page reference summarising the locked state of the project. Useful for stakeholder briefings and as a developer pocket reference.

Topic	Locked Decision
Roles	5 only — Super Admin, Lab In-Charge, Lab Engineer, Normal, View-Only (Admin DELETED)
equipment:create	Open to ALL roles except View-Only
Equipment lifecycle	Starts at PENDING_VERIFICATION → ACTIVE (BR-EQP-10)
Per-user role count	Exactly ONE primary role (BR-RBAC-02)
Super Admin seed	≥ 2 via SUPER_ADMIN_EMPLOYEE_IDS env var
Session model	15-min JWT · 60-min idle · 7-day refresh (concentric envelopes)
Tech stack (FE)	React 18 + Vite + Tailwind + Zustand + RQ + Axios + RHF + Zod
Tech stack (BE)	Express 4 + mysql2 + Zod + JWT + bcryptjs + Pino + PDFKit + Helmet + PM2
Stack additions (locked)	cookie-parser, compression, CSRF, react-table, tailwind-forms, vitest + supertest
Symmetry wins	Zod schemas + dayjs date math shared across FE / BE
Audit retention	Indefinite (until user specifies)
API base	/api/v1/... from day 1
DB approach	Raw SQL via mysql2/promise + Repository Pattern (no ORM)
PDF approach	pdfkit, streamed, never stored
Timeline	10 weeks (solo dev + AI pair); MVP first; full go-live after stakeholder sign-off

Topic	Locked Decision
DB target	~70 tables (existing 64 reviewed + new added in Phase 3)
MVP modules	Auth/RBAC · Equipment · Job Requests · Job Cards · Dashboard · Inquiry
Out of MVP	Schedule · Procurement · Reports · Master Data UI · Notifications
Deployment	On-prem · Nginx + PM2 + MySQL · no public cloud
Bottom line	Industry-grade, defence-grade context, demo-ready MVP in 10 weeks

25. The Very Next Step — Phase 3 Kickoff

Phase 3 begins the moment DS delivers the three open items (O1, O2, O3 from Section 22). The first conversation of Phase 3 will be:

```
PHASE 3 — DAY 1 AGENDA
======================

 1. DS shares the SHOW CREATE TABLE export
    or mysqldump --no-data for ~64 existing tables.

 2. DS provides the in-use / dead / unsure classification.

 3. DS provides the 2 Super Admin employee IDs.

 4. Audit:  cluster-by-cluster compare existing tables vs target.
            Produce a reconciliation table:
            keep · modify · drop · add  for every table.

 5. Lock the final schema for MVP-critical clusters first:
       a. Identity & Access  (cluster 1)
       b. Equipment Master   (cluster 3)
       c. Job Lifecycle      (cluster 4)
       d. Audit & Logs       (cluster 10)
       e. Lookups            (cluster 12)

 6. Defer Phase 2 cluster schemas (Schedule, Procurement,
    Reports, Notifications) — sketch only, not finalise.

 7. Write CREATE TABLE / ALTER TABLE scripts.

 8. Write seed scripts:
       - roles
       - permissions
       - role_permissions
       - super_admin users
       - lookup values

 9. Draw ER diagram for MVP-critical clusters.

 10. Stand up migration runner with versioned files.
```

⚠ IMPORTANT

Phase 3 is the most decision-dense phase of the project. Schema decisions, once deployed with real data, are expensive to reverse. We will spend the time to get this right — and we will do MVP-critical clusters first, deferring Phase 2 cluster details until they are needed.

Document Sign-off

This document represents the consolidated, locked, FINAL state of the CMCMIS_SIMPLIFIED project description prior to construction. It supersedes every interim document. Any future changes require an explicit new revision with a new version number — content of this document is not retroactively rewritten.

Document Version History

Version	Date	Status	Notes
v1.0 FINAL	Saturday, May 16, 2026	Locked	Phase 0 + 1 + 2 consolidated; all 20 decisions + 6 stack additions + 5 confirmations locked

Sign-off

Role	Name	Signature / Date
Prepared by	Deep Sorathiya (Software Developer Intern)	
Reviewed by		
Approved by		

Energy level: locked. Discipline level: locked.

Phase 0 → Phase 2 sealed. Phase 3 begins tomorrow morning.

— END OF FINAL DESCRIPTION —